LAPORAN PRAKTIKUM

- **Identitas Praktikum**

  Nama MK : Praktikum Pemrograman Berorientasi Objek

  Kode MK : CAK2KAB4

  Bobot SKS : 3 SKS

  Tempat : Lab MM

  Hari, tanggal : 25-04-2026

  Jam : 13.00 - selesai

  Topik praktikum : Modul- 07
- **Identitas Mahasiswa**

  Nama lengkap : DZAKI KHOTHIR

  NIM : 103112400113

  Program Studi : S-1 Teknik Informatika
- **Algoritma / studi kasus (jika ada) & Penjelasan**
- **Kode Program & Penjelasan**
- **Hasil Running Program**

Tugas Laporan Praktikum : Modul 6
Abstraksi dan Interface

SOAL CERITA

ShopKita adalah platform e-commerce lokal yang sedang mengembangkan fitur pembayaran. Platform ini ingin mendukung tiga metode pembayaran: dompet digital (seperti GoPay/OVO), kartu kredit, dan rekening bank.

Meskipun cara kerjanya berbeda, semua metode pembayaran harus bisa melakukan tiga hal: memproses pembayaran, mengecek saldo, dan menyimpan riwayat transaksi. Selain itu, setiap metode memiliki aturan validasi yang unik — dompet digital mengharuskan saldo mencukupi, kartu kredit tidak boleh melebihi limit, dan rekening bank memotong biaya admin sebesar Rp 2.500 setiap transaksi.

TUGAS MAHASISWA

- Buat interface Pembayaran dengan method bayar(), cekSaldo(), dan getRiwayat()
- Buat abstract class MetodePembayaran yang mengimplementasikan interface Pembayaran, dengan abstract method validasiPembayaran()
- Buat class DompetDigital, KartuKredit, dan RekeningBank yang masing-masing mewarisi MetodePembayaran
- Buat class Main untuk mensimulasikan minimal 2 transaksi per metode pembayaran (termasuk 1 transaksi yang gagal), lalu tampilkan riwayat seluruh transaksi

Aturan Validasi per Metode

Metode	Aturan Validasi
Dompet Digital	Saldo harus >= jumlah yang dibayar
Kartu Kredit	(Total tagihan + jumlah bayar) harus <= limit kredit
Rekening Bank	Saldo harus >= jumlah bayar + biaya admin Rp 2.500

Code Pembayaran.java

```
public interface Pembayaran {
    void bayar(double jumlah);
    double cekSaldo();
    void getRiwayat();
}
```

Code MetodePembayaran.java

```
import java.util.ArrayList;

public abstract class MetodePembayaran implements Pembayaran {
    protected double saldo;
    protected ArrayList<String> riwayat = new ArrayList<>();

    public MetodePembayaran(double saldo) {
        this.saldo = saldo;
    }

    // abstract method validasi
    protected abstract boolean validasiPembayaran(double jumlah);

    @Override
    public void bayar(double jumlah) {
        if (validasiPembayaran(jumlah)) {
            saldo -= jumlah;
            riwayat.add("Pembayaran berhasil: Rp " + jumlah);
            System.out.println("Pembayaran berhasil.");
        } else {
            riwayat.add("Pembayaran gagal: Rp " + jumlah);
            System.out.println("Pembayaran gagal.");
        }
    }

    @Override
    public double cekSaldo() {
        return saldo;
    }
```

```
        @Override
        public void getRiwayat() {
            for (String r : riwayat) {
                System.out.println(r);
            }
        }
    }
```

Code RekeningBank.java

```
    public class RekeningBank extends MetodePembayaran {
        private final double BIAYA_ADMIN = 2500;

        public RekeningBank(double saldo) {
            super(saldo);
        }

        @Override
        protected boolean validasiPembayaran(double jumlah) {
            return saldo >= (jumlah + BIAYA_ADMIN);
        }

        @Override
        public void bayar(double jumlah) {
            if (validasiPembayaran(jumlah)) {
                saldo -= (jumlah + BIAYA_ADMIN);
                 riwayat.add("Pembayaran berhasil: Rp " + jumlah + " +
admin");
                System.out.println("Pembayaran berhasil.");
            } else {
                riwayat.add("Pembayaran gagal: Rp " + jumlah);
                System.out.println("Pembayaran gagal.");
            }
        }
    }
```

Code KartuKredit.java

```
    public class KartuKredit extends MetodePembayaran {
        private double limit;
```

```
        private double tagihan;

        public KartuKredit(double limit) {
            super(0); // saldo ga kepake
            this.limit = limit;
            this.tagihan = 0;
        }

        @Override
        protected boolean validasiPembayaran(double jumlah) {
            return (tagihan + jumlah) <= limit;
        }

        @Override
        public void bayar(double jumlah) {
            if (validasiPembayaran(jumlah)) {
                tagihan += jumlah;
                riwayat.add("Pembayaran berhasil: Rp " + jumlah);
                System.out.println("Pembayaran berhasil.");
            } else {
                riwayat.add("Pembayaran gagal: Rp " + jumlah);
                System.out.println("Pembayaran gagal.");
            }
        }

        @Override
        public double cekSaldo() {
            return limit - tagihan;
        }
    }
```

Code DompetDigital.java

```
    public class DompetDigital extends MetodePembayaran {

        public DompetDigital(double saldo) {
            super(saldo);
        }

        @Override
        protected boolean validasiPembayaran(double jumlah) {
```

```
            return saldo >= jumlah;
        }
    }
```

Code Main.java

```
    public class Main {
        public static void main(String[] args) {

            DompetDigital dompet = new DompetDigital(100000);
            KartuKredit kartu = new KartuKredit(500000);
            RekeningBank bank = new RekeningBank(100000);

            dompet.bayar(50000);   // berhasil
            dompet.bayar(60000);   // gagal

            kartu.bayar(200000);   // berhasil
            kartu.bayar(400000);   // gagal (limit)

            bank.bayar(50000);     // berhasil
            bank.bayar(60000);     // gagal (saldo kurang + admin)

            System.out.println("\n=== RIWAYAT DOMPET ===");
            dompet.getRiwayat();

            System.out.println("\n=== RIWAYAT KARTU ===");
            kartu.getRiwayat();

            System.out.println("\n=== RIWAYAT BANK ===");
            bank.getRiwayat();
        }
    }
```

Output

```
PS C:\Users\dzaki\OneDrive\Dokumen\Praktikum PBO\Tugas Laprak\103112400113_Dzaki Khothir Modul 7\Source Code> cd "c:\Users\dzaki\OneDrive\Dokume
\Praktikum PBO\Tugas Laprak\103112400113_Dzaki Khothir Modul 7\Source Code\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
Pembayaran berhasil.
Pembayaran gagal.
Pembayaran berhasil.
Pembayaran gagal.
Pembayaran berhasil.
Pembayaran gagal.

=== RIWAYAT DOMPET ===
Pembayaran berhasil: Rp 50000.0
Pembayaran gagal: Rp 60000.0

=== RIWAYAT KARTU ===
Pembayaran berhasil: Rp 200000.0
Pembayaran gagal: Rp 400000.0

=== RIWAYAT BANK ===
Pembayaran berhasil: Rp 50000.0 + admin
Pembayaran gagal: Rp 60000.0
PS C:\Users\dzaki\OneDrive\Dokumen\Praktikum PBO\Tugas Laprak\103112400113_Dzaki Khothir Modul 7\Source Code>
```

Penjelasan

Object-Oriented Programming (OOP) dengan memanfaatkan interface, abstract class, dan inheritance untuk mensimulasikan sistem pembayaran pada platform e-commerce. Interface Pembayaran berfungsi sebagai kontrak yang mendefinisikan tiga method utama yaitu bayar(), cekSaldo(), dan getRiwayat(), yang wajib diimplementasikan oleh semua metode pembayaran. Selanjutnya, abstract class MetodePembayaran mengimplementasikan interface tersebut sekaligus menyediakan atribut umum seperti saldo dan riwayat transaksi, serta mendefinisikan method abstrak validasiPembayaran() yang harus diimplementasikan oleh subclass sesuai aturan masing-masing metode pembayaran.

Class turunan seperti DompetDigital, KartuKredit, dan RekeningBank mewarisi MetodePembayaran dan mengimplementasikan logika validasi yang berbeda sesuai dengan ketentuan yang diberikan. Pada DompetDigital, pembayaran hanya berhasil jika saldo mencukupi jumlah transaksi. Pada KartuKredit, sistem menggunakan konsep limit kredit, di mana transaksi akan ditolak jika total tagihan melebihi batas limit yang tersedia. Sementara itu, RekeningBank memiliki aturan tambahan berupa biaya administrasi, sehingga saldo harus mencukupi jumlah pembayaran ditambah biaya admin sebesar Rp2.500.

Selain itu, method bayar() pada abstract class digunakan untuk memproses transaksi dengan terlebih dahulu melakukan validasi. Jika validasi berhasil, saldo akan dikurangi dan transaksi dicatat sebagai berhasil, sedangkan jika gagal akan dicatat sebagai transaksi

gagal. Semua riwayat transaksi disimpan dalam struktur data ArrayList dan dapat ditampilkan menggunakan method getRiwayat(). Pada class Main, dilakukan simulasi beberapa transaksi untuk setiap metode pembayaran, termasuk transaksi yang berhasil dan gagal, kemudian seluruh riwayat transaksi ditampilkan sebagai output program. Dengan desain ini, program menunjukkan penerapan prinsip abstraksi, enkapsulasi, dan polimorfisme dalam pengembangan sistem yang modular dan mudah dikembangkan.